Augie's Rentals Ops

A one-operator tool-rental business running on a home server, a laptop that hosts every language model, and a public storefront stitched together from Booqable, Squarespace and Cloudflare. This is the tour for developers joining from other projects — what it does, how it is put together, where each model runs, and how to measure what those models actually cost per unit of work.

~70k LINES PYTHON	**15k** LINES VANILLA JS	**70** POSTGRES TABLES	**83** FORWARD-ONLY MIGRATIONS
102 TEST FILES	**13** AI MODULES	**0** BUILD STEPS	

ORIENTATION

What the system is

> Not a product. A private operations system for one business, where a bad deploy does not turn a build red — it rewrites 84 live product descriptions, or puts a wrong number in a CRA filing.

Augie's Rentals rents tools, trailers and moving kits in Telkwa, British Columbia. The commercial storefront is augiesrentals.ca — a Squarespace shell wrapped around Booqable's rental-commerce embeds. This repository is everything behind it: the catalogue automation that writes to Booqable, a Postgres database that is the source of truth for inventory, finance, tax and incidents, a FastAPI admin app the owner runs the business from, and the local-AI tooling that drafts copy, reads receipts and classifies risk.

Three properties shape almost every design decision, and they are the parts worth carrying to another project:

1. **Every write is dry-run by default.** Not by convention — every write-capable script reads a `BOOQABLE_APPLY_*` environment variable that the launcher sets only when `--apply` is passed. There is no Make target for live writes.

2. **Every AI output is advisory.** No model writes to the catalogue, the ledger or the storefront — a suggestion reaches production only when a human accepts it on a screen. There is exactly one AI-written table in the schema, a derived presentation cache for search insights, and it is documented as the deliberate exception it is. This is what makes the whole system safe to build on models that are sometimes wrong — and, usefully, it also gives you a free quality metric (see Measurement).
3. **The models live on a laptop that is often asleep.** Inference is a network call to a MacBook over WireGuard. Every AI feature therefore has a designed offline state and a stated manual fallback. "AI unavailable" is the normal condition for most of the day, not an incident.

ARCHITECTURE

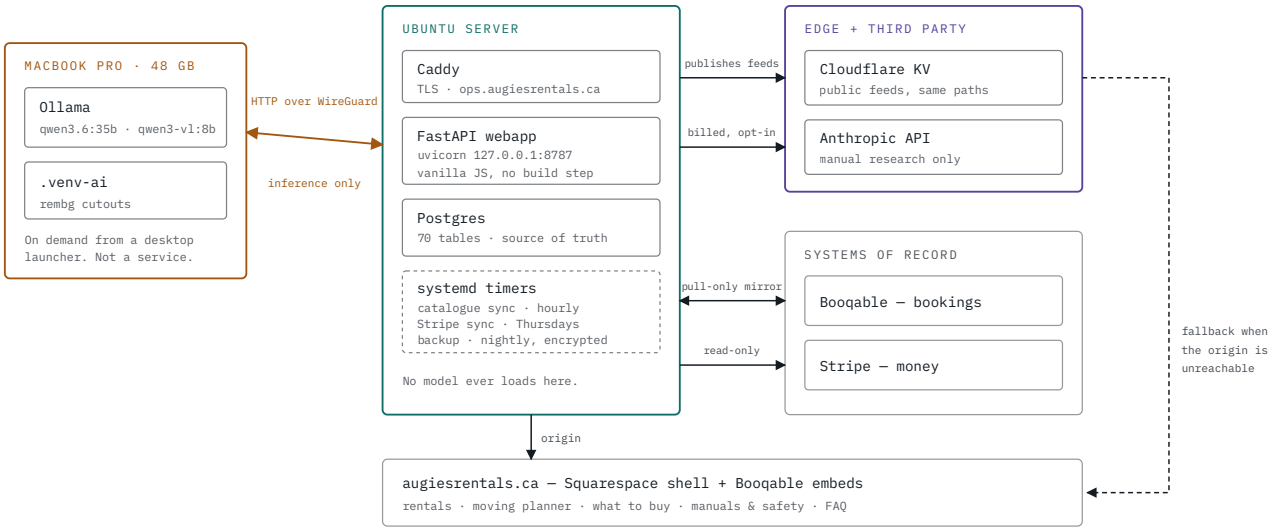
Where the code physically runs

Four runtimes, and which one a piece of work lands on is a rule, not a preference. The rule: **the server runs the app, the Mac runs every model, the edge serves the public feeds.**

MAC operator's MacBook Pro (M5 Pro, 48 GB) — Ollama only, on demand

SERVER Ubuntu box in a closet — webapp, Postgres, timers

CLOUD billed third-party API or edge worker



The Mac is the only machine that loads a model, and it reaches the system rather than the other way round. The dashed Cloudflare path is the one that keeps the public storefront alive through a power cut in Telkwa: the server publishes its own public API responses into KV after each catalogue sync, under the same paths, so the page falls back with a base-URL swap.

Why inference is not on the server

It looks backwards — the always-on machine calls the sometimes-asleep one — and it is deliberate. The server is a repurposed laptop in a closet; the Mac has 48 GB of unified memory and the models. Putting a model environment on the server would mean maintaining `rembg`, PyMuPDF and a GPU-less inference stack there for work that would run several times slower.

The consequence is a placement rule that gets enforced in code review and in tests: **server-side AI features keep the cheap work local and send only the inference over the wire.** Receipt scanning is the canonical example — Pillow decodes and downscales the photo on the server, `pypdf` pulls a PDF's text layer on the server, and only a base64'd image goes to the Mac as a plain `requests` call. A test parses imports to assert that the incident-reporting path has no AI dependency at all, because an incident happens when it happens and cannot wait for a laptop to wake up.

STACK

What it is built from

LAYER	CHOICE	THE REASON IT IS THAT
Backend	`Python 3.14, FastAPI, uvicorn`	Routes live one-domain-per-file under `webapp/backend/routes/`; auth is middleware on the `/api/` prefix, so a new route is protected by default.
Frontend	`vanilla JS, hash router`	No build step, no framework, no node_modules. Plain `<script>` tags. One file per page. The whole app is editable over SSH on a bad connection.
Database	`Postgres 16 (Docker)`	No ORM, no Alembic. Numbered forward-only `.sql` files applied by a 200-line migrator tracking a `schema_migrations` table.
Inference	`Ollama over HTTP`	One client module, `scripts/lib/ollama_client.py`. The `ollama` Python package is allowed only inside `.venv-ai` on the Mac.
Commerce	`Booqable API v4`	Single live catalogue, no sandbox — so writes are blocked by a `requests.Session` subclass outside production rather than redirected.
Tests	`pytest + Playwright`	Mostly browser-level against a running webapp; markers are derived at collection (`browser` / `server` / `pure`), never hand-applied.
Edge	`Cloudflare Worker + KV`	Serves the storefront's public feeds from a global edge so a home power cut does not remove the rentals browser from the website.

Two Python environments, deliberately separated. `.venv` runs everything: the CLI, the Booqable scripts, the webapp. `.venv-ai` is a heavier environment (rembg, PyMuPDF, onnxruntime) that exists only on the Mac for the three jobs that genuinely need model weights on local disk. Scripts in `.venv` that need `.venv-ai` shell out to it per item rather than importing across environments.

FUNCTIONS

What the system actually does

The admin app organises itself around *benches* — where the work physically happens — rather than around the database schema. That is the single most transferable UI idea in the repo: the navigation is a data file, and every destination carries aliases for the names it used to have, so nobody has to remember what a screen got renamed to.

BENCH	CADENCE	WHAT LIVES THERE
Yard	daily, phone-first	Today's pickups and returns, rentals, find a tool, add a receipt, log a trip, report an incident, Ask AI.
Books	weekly → monthly	Approve Stripe charges, expenses, income, bank reconciliation, P&L reports, capital assets.
Storefront	as the catalogue changes	Catalogue pipeline, put a tool or bundle online, kit photos, safety warnings, what-to-buy guides, manuals, FAQ, search analytics, tow checks, launch audit.
Analytics	campaign-driven	Trackable links and QR codes, email outreach with open tracking, booking attribution.
Tax	seasonal	T2125 walkthrough, CCA schedules, vehicle mileage — sole-proprietorship filing, every figure computed in Python.

The interesting subsystems

NEW TOOL INTAKE

Photo → background removal → photo QA → research → copy → Booqable create. The wizard's pure functions all take a plain field-keyed dict, so they are storage-agnostic; live publishing needs a typed `APPLY` confirmation on top of the `--apply` flag.

STRIPE REVIEW QUEUE

A weekly timer pulls succeeded charges read-only, matches each to the Booqable order its description names, and stages it as `pending`. That match is the only place the system gets a GST/PST split at all — Stripe reports one lump total. Nothing enters the ledger until the owner approves it on screen.

INCIDENT CAPTURE

Damage, injury, theft or complaint, written at the roadside. Two required fields. Every later edit lands in a revisions table in the same transaction, empty always means "not filled in yet" rather than "erase", and the whole path is deliberately free of any AI dependency.

MOVING BUNDLE PLANNER

Builds a customer's Booqable cart from pre-built bundles that all draw from one shared tote pool. The constraint the whole page is designed around: a Booqable order has exactly one rental period, so the trailer must never ride the tote order — you pack for weeks, you move for days.

STOREFRONT SEARCH ANALYTICS

Anonymous capture of what visitors type into the rentals search bar — no IP, no cookies, a random per-tab key. One unauthenticated POST endpoint treated as hostile input, with hard caps and rate limits, feeding a gap analysis the owner reads.

TAX LAYER

Store judgement, compute only mechanics. CCA class, fair market value and how much to claim are operator inputs because each is a position someone has to defend; only declining-balance arithmetic is derived. A reconcile function re-derives every total from raw rows because two summaries once disagreed by $1,118.83 and nothing surfaced it.

The public storefront

Squarespace holds only a stub. An embed loader fetches each widget from this repo's server (or Cloudflare, on fallback): the rentals browser with category filters and live price ladders, the moving planner, the what-to-buy guides, the equipment manuals and safety page, and the FAQ. All prices shown to a customer are Booqable's own, read live — the config files carry fallbacks, never a second source of truth.

MODELS

Every model, and what it is for

> Local by default. Two documented exceptions, each with a written reason, an opt-in key, and a degradation path when the key is blank.

MODEL	RUNS	ENV VAR	USED BY	RESIDENT
`qwen3.6:35b-a3b` MoE, ~3B active	MAC	`OLLAMA_CHAT_MODEL`	Inventory chat, FAQ tone review, expense category check, consumables drafts, search-gap and tow-check analysis, purchase price research, bundle copy	~22 GB
`qwen3:30b-instruct` MoE, ~3B active	MAC	`OLLAMA_COPY_MODEL`	Storefront copy, new-tool research, risk classification, competitor pricing research	~19 GB
`qwen3-vl:8b-instruct` dense vision	MAC	`OLLAMA_VISION_MODEL`	New-tool photo QA (is this hero shot clean and centred?)	~7 GB
`qwen3-vl:8b-instruct` via its own var	MAC SERVER	`OLLAMA_RECEIPT_MODEL`	Receipt scanning. Has its own variable so it can diverge; never falls back to a text model, which would be blind to a photo	~7 GB
`nomic-embed-text` ~140M params	SERVER	`OLLAMA_EMBED_MODEL`	Catalogue search and the Ask AI knowledge base. Has its own host variable (`OLLAMA_EMBED_HOST`) so it can run on the server: a customer searching at 9pm on a Sunday cannot depend on a laptop being awake	~0.3 GB
`birefnet-general` rembg, not Ollama	MAC	`BACKGROUND_REMOVAL_MODEL`	Product photo background removal, cross-checked against u2net. The one job that genuinely needs weights on local disk	small
`claude-opus-5` effort xhigh, web search	CLOUD	`ANTHROPIC_API_KEY`	Finding a real, currently-live manufacturer manual URL. Every URL it returns is independently HTTP-verified before it is trusted	–
`Gemini` Apps Script	CLOUD	`GEMINI_API_KEY`	Google Drive photo renamer. Runs on Google's infrastructure and cannot reach a local Ollama server	–

The two text variables have drifted a little in practice: one module defaults to the chat model while naming `OLLAMA_COPY_MODEL` in its error hint. Nothing is broken by it, and nothing surfaces it either — which is a small illustration of the argument in the next section. You cannot see which model actually served a feature until you record it.

THE RAM BUDGET IS A REAL CONSTRAINT

All three Ollama defaults resident at once is roughly 45–50 GB on a 48 GB machine that is also running macOS. Ollama evicts the least-recently-used model, so a session alternating copy generation and chat pays a full 30–60 second reload on every switch — exactly the cost `keep_alive` was added to remove. The documented fix is to point `OLLAMA_COPY_MODEL` at the same model as `OLLAMA_CHAT_MODEL`: the two variables stay separate so upgrading one never silently changes the other, but nothing stops them naming the same thing.

Rules that govern where a model call may live

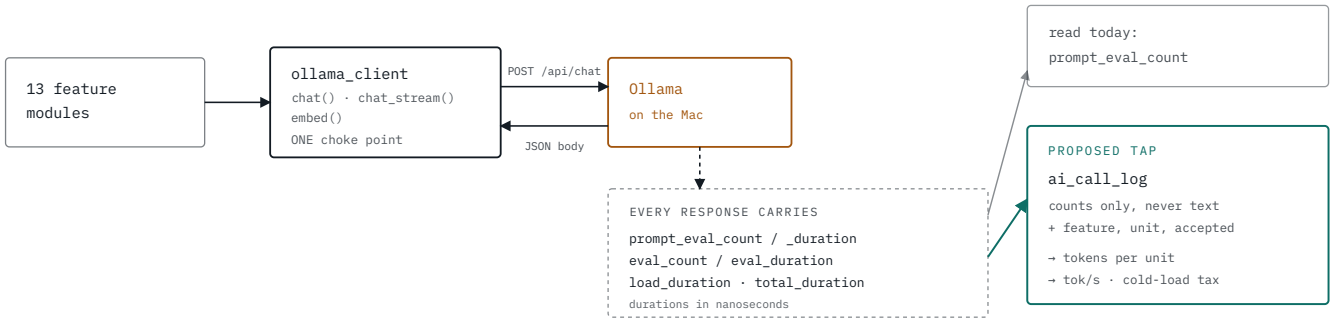
- **Nothing under `scripts/local_ai/` may be imported by the webapp.** That directory runs inside `.venv-ai` on the invoking machine; importing it from a server-side feature quietly demands a model environment on the server. Server-side AI modules live in `scripts/lib/` and speak plain HTTP.
- **Prompts and schemas that decide the same thing must be byte-identical.** Risk classification exists in two places for placement reasons; a test asserts the prompts match, because it decides the safety wording a customer reads.
- **Fetched web content and visitor-typed text are evidence, never instructions.** Search terms, receipt text and scraped pages all get this treatment explicitly.
- **Every model answer is re-decided in Python.** A receipt's category must match a closed set enforced by a database `CHECK` constraint, money must parse as money, a missing subtotal is computed rather than asked for. Business-use percentage and capital-asset status are never extracted at all — they are the owner's judgement about their own business, not facts on a receipt.
- **Degradation is designed, not incidental.** Each feature declares what it needs (`mac` / `server` / `api`) and the fallback sentence shown when it is unavailable. Controls that cannot work are disabled with the fallback printed next to them, rather than left pressable to fail on click.

MEASUREMENT

Quantifying what the models cost

> Ollama returns per-call counters on every response. This repo currently reads exactly one of them and throws the rest away. Those counters are the entire measurement.

Today `ollama_client.check_context_fit()` pulls `prompt_eval_count` out of the response body to warn when a prompt has filled its context window, then discards the body. Sitting unread in that same JSON are the numbers that answer "what does this feature cost?" — and because there is exactly one HTTP client for every server-side AI feature, there is exactly one place to tap.



One client module means one instrumentation point. Every server-side AI feature already funnels through `chat()`, `chat_stream()` and `embed()`, and each of those already parses the response body — the counters are in hand and are being dropped on the floor.

The metric set

Tokens per unit is the number that ports between platforms. Tokens are a property of the task and the model; seconds are a property of the hardware; dollars are a property of the vendor. Measure tokens per unit of business work and you can price the same feature on a laptop, a server or an API without re-running anything. Everything else is derived from it and from the durations.

METRIC	FORMULA	WHAT IT TELLS YOU
Tokens per unit	`(Σ prompt + Σ output) ÷ units`	The portable cost of one receipt read, one tool researched, one FAQ reviewed. Multiply by any vendor's rate to price the same feature on a cloud API.
Generation throughput	`eval_count ÷ eval_duration`	Tokens per second while writing. The number that decides whether a feature feels instant or feels like waiting.
Prefill throughput	`prompt_eval_count ÷ prompt_eval_duration`	Tokens per second while reading the prompt. Dominates long-context features; irrelevant to short ones.
Wall seconds per unit	`total_duration ÷ units`	What the operator actually experiences. Includes queueing and load.
Cold-load tax	`load_duration ÷ total_duration`	The share of a call spent loading the model. On this hardware a cold call can be 8× a warm one — it is the single biggest lever, and it is a config change, not a model change.
Context headroom	`prompt_eval_count ÷ num_ctx`	How close a prompt is to silent front-truncation. Already warned about; worth trending, because prompts grow with the catalogue.

METRIC	FORMULA	WHAT IT TELLS YOU
Acceptance rate	`accepted ÷ suggested`	**The quality metric.** Every AI output here is advisory, so the operator pressing "use this" or editing it first is a free, continuous, real-world quality signal. No eval harness needed.
Edit distance	`Levenshtein(suggested, saved)`	How much of an accepted suggestion survived. Separates "right idea, wrong wording" from "accepted because dismissing it is friction".

THROUGHPUT IS THE CHEAP METRIC; ACCEPTANCE IS THE REAL ONE

A model that is twice as fast and gets rejected twice as often has made the business slower. Because this system never lets a model write directly, acceptance rate is already observable at every write point — it needs recording, not inventing. Any team building human-in-the-loop AI should wire this up before they wire up a benchmark.

The tap: one table, one hook, no prompt text

One row per model call. **Counts only — never prompt or completion text.** These calls carry receipts, incident reports and customer names; a metrics table that stores content becomes a second, unaudited copy of the most sensitive data in the system.

```
-- db/migrations/1NN_ai_call_log.sql
CREATE TABLE ai_call_log (
  id              uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  occurred_at     timestamptz NOT NULL DEFAULT now(),

  feature         text    NOT NULL,   -- 'receipt_ocr', 'faq_tone', 'search_gaps'
  model           text    NOT NULL,
  host_role       text    NOT NULL,   -- 'mac' | 'server' | 'api'
  endpoint        text    NOT NULL,   -- 'chat' | 'chat_stream' | 'embed' | 'anthropic'

  prompt_tokens   integer,
  output_tokens   integer,
  cached_tokens   integer,            -- Anthropic cache reads; null for Ollama
  num_ctx         integer,

  load_ms         numeric(10,2),
  prefill_ms      numeric(10,2),
  generate_ms     numeric(10,2),
  total_ms        numeric(10,2),

  unit_kind       text,               -- 'receipt' | 'tool' | 'faq_entry' | 'report'
  unit_id         text,               -- ties a multi-call job back to one unit of work
  ok              boolean NOT NULL,
  error_class     text,               -- 'unreachable' | 'model_missing' | 'bad_response'
  accepted        boolean,            -- set later, when the operator saves or discards
  edit_distance   integer             -- set with `accepted`
);

CREATE INDEX ai_call_log_feature_time ON ai_call_log (feature, occurred_at DESC);
CREATE INDEX ai_call_log_unit         ON ai_call_log (unit_kind, unit_id);
```

The hook goes in `ollama_client.chat()`, which already has the parsed body in hand for the truncation check:

```
# scripts/lib/ollama_client.py — inside chat(), where check_context_fit already runs
body = resp.json()
check_context_fit(body, _num_ctx(num_ctx))
record_call(                        # no-op when AI_METRICS is off
    feature=feature, model=model, host=target, endpoint="chat",
    body=body, num_ctx=_num_ctx(num_ctx), unit_kind=unit_kind, unit_id=unit_id,
)
return ((body.get("message") or {}).get("content") or "").strip()
```

Three properties keep this from becoming a liability of its own:

- **It must never fail a feature.** Recording is wrapped so a database hiccup logs and moves on. A metrics table that can break receipt scanning is worse than no metrics.
- **Callers pass `feature` and `unit_id`, nothing else.** One extra keyword per call site. Fourteen modules, one line each.
- **Streaming needs the final chunk.** `chat_stream()` already captures the terminal `done` body for `check_context_fit` — the counters are in that same object.

The Anthropic path already computes the equivalent and prints it: `claude_manual_research.py` accumulates `input_tokens`, `output_tokens`, `cache_creation_input_tokens` and `cache_read_input_tokens` across the agent loop. That total goes into the same table with `host_role = 'api'`, which is what makes the local and billed paths directly comparable per unit.

The bench harness

The log tells you what production costs. A benchmark tells you what a *different* model or host would cost, before you switch. `ai_bench.py` replays a fixed corpus of five tasks — each shaped like a real feature in this repo — against any set of models and any set of Ollama hosts, and reports medians over warm runs with cold loads called out separately.

```
# compare two models on this Mac
python3 ai_bench.py --models qwen3.6:35b-a3b,qwen3:30b-instruct --repeats 3

# compare the same model on two machines — this is the "platforms" question
python3 ai_bench.py --host http://localhost:11434 \
                    --host http://10.10.0.2:11434 \
                    --models qwen3-vl:8b-instruct --tasks extract,embed

# machine-readable, for trending
python3 ai_bench.py --json bench.json --markdown bench.md
```

The corpus is synthetic on purpose — a fake Home Hardware receipt, a fake cut-off saw, 420 generated search terms — so it is safe to commit, safe to share, and identical on every machine it runs on. Comparing platforms only means anything if the input is byte-identical.

BASELINE

Measured, not estimated

The harness run below is real: five tasks against the three Ollama models actually installed on the operator's Mac, two repetitions each, medians over warm runs.

TASK	MODEL	PROMPT	OUTPUT	TOKENS PER UNIT	PREFILL TOK/S	GENERATION TOK/S	WARM S	COLD S	
Risk classification structured · 80-token prompt	qwen3-vl:8b-instruct	82	65	**147**	636	43	1.7	4.0	
	qwen3.6:35b-a3b	86	82	**168**	440	60	1.6	12.8	
	qwen3:30b-instruct	82	1,270	**1,352**	516	68	9.6	33.3	output varied 646–1894

TASK	MODEL	PROMPT	OUTPUT	TOKENS PER UNIT	PREFILL TOK/S	GENERATION TOK/S	WARM S	COLD S	
Receipt extraction structured · receipt text	qwen3-vl:8b-instruct	253	75	**328**	940	42	1.9	3.2	
	qwen3.6:35b-a3b	254	81	**335**	842	57	1.6	8.5	
	qwen3:30b-instruct	253	76	**329**	958	71	1.2	5.4	
Storefront copy prose · ~120 words	qwen3-vl:8b-instruct	101	122	**224**	812	43	3.0	—	
	qwen3.6:35b-a3b	105	150	**254**	551	60	2.8	—	
	qwen3:30b-instruct	101	142	**243**	686	69	2.2	—	
Search gap analysis structured · ~4k-token prompt	qwen3-vl:8b-instruct	3,869	178	**4,047**	667	36	6.0	11.3	
	qwen3.6:35b-a3b	4,272	78	**4,350**	1,057	54	1.7	12.5	
	qwen3:30b-instruct	3,869	1,204	**5,073**	782	50	33.9	33.9	`cut off at 900 s`
Knowledge embedding batch of 32 chunks	nomic-embed-text	896	0	**896**	4,240	—	0.2	—	

Median of two runs per cell on an M5 Pro MacBook Pro, 48 GB, Ollama 0.32.14, 2026-09-01. Generation rate is from warm runs; prefill rate is from the *first* run of each cell only, for the reason in the third finding below.

What this run actually showed

TOKENS PER UNIT SEPARATED THE MODELS; TOKENS PER SECOND DID NOT

On risk classification, `qwen3.6:35b-a3b` answers in 84 output tokens. `qwen3:30b-instruct` spent 646 tokens on one run and 1,894 on the other — the same prompt, the same schema, up to **22× the output** for an answer that is three fields long. Generation throughput points the other way: the 30B model is the *faster* of the two per token (68 vs 60 tok/s). Rank these models on tok/s and you pick the one that costs an order of magnitude more per unit of work.

The same task pushed further on the long-context analysis. The 35B answered in 81 output tokens and 1.7 seconds warm. The 30B produced 1,204 tokens on its first pass, then on an identical second pass ran past the harness's 900-second read timeout and was cut off. Because the call never returned, the output itself was never captured — a

repetition loop under the structured-output constraint is the likely cause, not a confirmed one. What is certain is that the call was still generating at 68% CPU fourteen minutes in. Nothing in this repo's current instrumentation would show that; the operator would experience it as "the button is still spinning".

THE COLD-LOAD TAX IS THE BIGGEST LEVER, AND IT IS A CONFIG SETTING

Risk classification on the 35B: 12.8 seconds cold, 1.6 seconds warm. Eight times, and none of it is inference — it is the model being read into memory. Every cold column in the table is the cost of an eviction, which is the RAM-budget problem from the previous section showing up as latency. Consolidating the two text variables onto one model buys more than any model upgrade would.

A REPEATED IDENTICAL PROMPT DOES NOT MEASURE PREFILL

Ollama caches the processed prompt. The second run of the long-context task reported 89,518 and 115,005 prefill tokens per second — not a measurement, a cache hit, against a genuine first-pass rate of roughly 700–1,100 tok/s. Any benchmark that replays one prompt and averages will overstate prompt-processing by about two orders of magnitude. This is why the table reports first-pass prefill, and why a harness meant for trending should vary its inputs.

EMBEDDINGS ARE FREE, WHICH IS WHAT MAKES THE SPLIT HOST POSSIBLE

32 chunks — 896 tokens — embedded in 0.2 seconds at roughly 4,900 tok/s, on a model that occupies 0.3 GB. That is the quantitative case for `OLLAMA_EMBED_HOST` pointing somewhere other than the laptop: catalogue search costs so little that it can run on the server, and a customer searching the storefront then never depends on a laptop being awake.

WHAT A TEAM SHOULD TAKE FROM THIS

Five tasks, three models, about thirty minutes of wall clock, and it surfaced a runaway generation, an 8× cold-start penalty and a benchmarking trap. None of those are visible in a throughput number, and none of them required a quality eval. Record the counters you are already being handed, express the result per unit of work, and most of the decisions make themselves.

PRACTICES

The parts worth stealing

Most of what makes this repo unusual is process rather than code, and it exists because the failure modes here are quiet. A wrong tax figure does not throw. Eighty-four rewritten product descriptions do not fail a build.

DRY-RUN IS A MECHANISM, NOT A HABIT

The launcher owns the mapping from `--apply` to per-command environment variables. A script cannot accidentally write live because it never sees the switch unless the launcher set it, and live writes are deliberately not reachable from `make`.

YOU MAY NOT REVIEW YOUR OWN WORK

Every non-documentation change needs a review by a session that did not write it — not a subagent, not the same window after a reset. Higher-risk changes additionally go to a different model entirely, on the reasoning that a fresh Claude window is still Claude, with the same priors that wrote the bug.

A GUARD IS NOT DONE UNTIL YOU HAVE WATCHED ITS TEST FAIL

Written after a review where two confident claims would have been disproved by a single grep. A behavioural claim names the command that showed it, or says "not verified".

REDESIGN ON THE SECOND FINDING

If a bug returns in a new guise, remove the mechanism that allows it rather than patching the new case. This rule came out of an eight-round review where four rounds patched instances of one root cause.

COMPUTED VALUES ARE NEVER STORED

Suggested rates, deposits, risk ratings and readiness flags are recomputed on every read. There is exactly one implementation of each business rule and it cannot drift out of sync with a cached copy.

NO HARD DELETES

Unpublish, void, supersede. The change log is a page in the app, and reversing something lands as a new line rather than an erasure — which is also what the books require.

`augie15/augies-rentals-ops · private · single-operator deployment · this brief describes architecture and measurement, and contains no credentials, customer data or business figures`